

Reward Design Function - análisis visual preservando el espíritu del notebook anterior

Este notebook vuelve al objetivo del diseño original: entender la reward **visualmente** con mapas de calor, perfiles y distribuciones de comportamiento conjunto.

La diferencia es que ahora todo queda alineado con la implementación vigente del sistema:

- la reward principal nace de métricas interval-level `L_*_<var_obj>`
- los extras activos son `bandwidth_bonus` y `dynamic_penalty`
- la asignación final es por lazo (`controller_reward`)

Para conservar la intuición visual del notebook previo, se usan dos vistas complementarias:

1. **vista exacta en el espacio de métricas:** mapas sobre `L_e`, `L_edot`, `L_delta_u`
2. **vista proxy en el espacio de comportamiento:** mapas sobre `|error|` y `|control_action|`

Esa segunda vista asume intervalos cuasi estacionarios, lo que permite proyectar la reward actual sobre ejes comparables a los que motivaban el notebook anterior.

```
In [1]: from pathlib import Path
import sys

import yaml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

try:
    from IPython.display import display, Markdown
except ImportError:
    def display(obj):
        print(obj)

    def Markdown(text):
        return text

ROOT = Path.cwd()
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from metrics.metric_processing import MetricProcessing
from rewards.reward_calculator_base import RewardCalculatorBase

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams["figure.dpi"] = 120
```

```

CONFIG_PATH = ROOT / "config" / "config_CartPole.yaml"
with CONFIG_PATH.open("r", encoding="utf-8") as f:
    config_main = yaml.safe_load(f)

simulation_cfg = config_main["simulation"]
reward_base_cfg = config_main["reward_base"]
reward_cfg = reward_base_cfg["reward_config"]
reward_calc_cfg = reward_base_cfg["reward_calculation"]
principal_cfg = reward_calc_cfg["principal_reward"]
extra_cfg = reward_calc_cfg["extra_rewards"]
normalization_cfg = reward_calc_cfg["metric_processing"]["normalization"]

metric_processing = MetricProcessing(config_main)
reward_calculator = RewardCalculatorBase(config_main)

active_var_objs = list(dict.fromkeys(reward_calculator.agent_to_var_obj_map.values()))
decision_steps = int(simulation_cfg["decision_interval_sec"] / simulation_cfg["decision_interval_sec"])

if principal_cfg["method"] == "lineal_combination":
    principal_features = principal_cfg["lineal_combination_params"]["features"]
else:
    principal_features = principal_cfg["weighted_exponential_params"]["features"]

band_cfg = extra_cfg["bonus_approach"]["bandwidth_bonus"]
dyn_pen_cfg = extra_cfg["conditional_approach"]["dynamic_penalty"]

print(f"Config cargada desde: {CONFIG_PATH}")
print(f"reward approach = {reward_cfg['reward_approach']}")
print(f"principal method = {principal_cfg['method']}")
print(f"loops activos = {active_var_objs}")
print(f"steps por intervalo = {decision_steps}")

```

Config cargada desde: c:\Users\davor\Escritorio\DOCTORADO\Tesis\RL_5_SimpleSystem Simulator\config\config_CartPole.yaml
reward approach = controller_reward
principal method = weighted_exponential
loops activos = ['pendulum_angle', 'cart_position']
steps por intervalo = 20

```

In [2]: def get_range_abs(var_obj, feature_key):
        cfg = normalization_cfg["params"][var_obj][feature_key]
        lo, hi = cfg["range"]
        return max(abs(lo), abs(hi))

def normalized_metric_from_constant_level(var_obj, feature_key, level):
    cfg = normalization_cfg["params"][var_obj][feature_key]
    method = cfg["method"]
    lo, hi = cfg["range"]
    out_lo, out_hi = normalization_cfg["output_limits"]

    arr = np.asarray(level, dtype=float)
    abs_arr = np.abs(arr)

    if method == "mean_squared":
        span = max(abs(lo), abs(hi)) ** 2
        normalized_01 = np.square(abs_arr) / span if span > 0 else np.zeros_like(abs_arr)
    elif method in {"abs", "rms"}:
        span = max(abs(lo), abs(hi))
        normalized_01 = abs_arr / span if span > 0 else np.zeros_like(abs_arr)

```

```

else:
    span = hi - lo if hi != lo else 1.0
    normalized_01 = np.abs(arr - lo) / span

normalized_01 = np.clip(normalized_01, 0.0, 1.0)
return out_lo + normalized_01 * (out_hi - out_lo)

def principal_loop_reward(var_obj, error_level, edot_level, I_level=0.0, delta_u
    L_e = normalized_metric_from_constant_level(var_obj, "e", error_level)
    L_edot = normalized_metric_from_constant_level(var_obj, "edot", edot_level)
    L_I = normalized_metric_from_constant_level(var_obj, "I", I_level)
    L_delta_u = normalized_metric_from_constant_level(var_obj, "delta_u", delta_u

    if principal_cfg["method"] == "lineal_combination":
        return (
            -principal_features["L_e"]["weight"] * L_e
            -principal_features["L_edot"]["weight"] * L_edot
            -principal_features["L_I"]["weight"] * L_I
            -principal_features["L_delta_u"]["weight"] * L_delta_u
        )

    result = np.zeros_like(np.asarray(L_e, dtype=float))
    values = {
        "L_e": L_e,
        "L_edot": L_edot,
        "L_I": L_I,
        "L_delta_u": L_delta_u,
    }
    for feature_name, value in values.items():
        params = principal_features[feature_name]
        result -= params["weight"] * (
            1.0 - np.exp(-params["scaled"] * (value - params.get("setpoint", 0.0
        )
    return result

def get_band_range(var_obj):
    return band_cfg["ranges"][f"error_{var_obj}"]

def band_bonus_interval(var_obj, error_level, accumulated=0.0):
    band_lo, band_hi = get_band_range(var_obj)
    error_abs = np.abs(np.asarray(error_level, dtype=float))
    band_limit = max(abs(band_lo), abs(band_hi))
    in_band = error_abs <= band_limit
    raw_bonus = np.where(in_band, decision_steps * band_cfg["per_step_band_bonus
    max_total = band_cfg["max_total_band_bonus"][var_obj]
    return np.minimum(raw_bonus, np.maximum(0.0, max_total - accumulated))

def dynamic_penalty_interval(var_obj, control_action_level, error_level):
    if not dyn_pen_cfg["enabled"]:
        return np.zeros_like(np.asarray(control_action_level, dtype=float))

    signal_name = f"control_action_{var_obj}"
    cfg = dyn_pen_cfg["dynamic_penalty_params"].get(signal_name)
    if cfg is None:
        return np.zeros_like(np.asarray(control_action_level, dtype=float))

```

```

signal_abs = np.abs(np.asarray(control_action_level, dtype=float))
error_abs = np.abs(np.asarray(error_level, dtype=float))
cond = cfg["condition"]
cond_factor = np.exp(-cond.get("scaled", 1.0) * (error_abs - cond.get("setpc

if dyn_pen_cfg["method"] == "lineal":
    return -cfg["weight"] * signal_abs * cond_factor
return -cfg["weight"] * (signal_abs ** 2) * cond_factor

def total_loop_reward_proxy(var_obj, error_level, control_action_level, edot_level,
    principal = principal_loop_reward(var_obj, error_level, edot_level, I_level,
    band = band_bonus_interval(var_obj, error_level)
    dyn_pen = dynamic_penalty_interval(var_obj, control_action_level, error_level)
    return principal + band + dyn_pen

def plot_heatmap(ax, x_axis, y_axis, z_values, x_label, y_label, title, cmap, cc
    image = ax.imshow(
        z_values,
        extent=[x_axis.min(), x_axis.max(), y_axis.min(), y_axis.max()],
        origin="lower",
        aspect="auto",
        cmap=cmap,
    )
    ax.set_xlabel(x_label)
    ax.set_ylabel(y_label)
    ax.set_title(title)
    plt.colorbar(image, ax=ax, label=colorbar_label)

def build_summary_tables():
    principal_df = pd.DataFrame(principal_features).T.rename_axis("feature").reset_index()
    extra_status_df = pd.DataFrame(
        [
            {"block": "bandwidth_bonus", "enabled": band_cfg["enabled"]},
            {"block": "dynamic_penalty", "enabled": dyn_pen_cfg["enabled"]},
            {"block": "dynamic_incentive", "enabled": extra_cfg["conditional_app"},
            {"block": "goal_bonus_reward", "enabled": extra_cfg["bonus_approach"]},
            {"block": "penalty_delta_var", "enabled": extra_cfg["penalty_approach"]},
            {"block": "penalty_instantaneous_reward", "enabled": extra_cfg["penalty_instantaneous_reward"]}
        ]
    )
    band_df = pd.DataFrame(
        [
            {
                "var_obj": var_obj,
                "band_abs_max": abs(get_band_range(var_obj)[1]),
                "per_step_bonus": band_cfg["per_step_band_bonus"],
                "interval_bonus_if_full_band": decision_steps * band_cfg["per_step_band_bonus"],
                "max_total_band_bonus": band_cfg["max_total_band_bonus"][var_obj]
            }
        ]
    )
    for var_obj in active_var_objs:
        band_df.loc[len(band_df)] = {
            "var_obj": var_obj,
            "band_abs_max": abs(get_band_range(var_obj)[1]),
            "per_step_bonus": band_cfg["per_step_band_bonus"],
            "interval_bonus_if_full_band": decision_steps * band_cfg["per_step_band_bonus"],
            "max_total_band_bonus": band_cfg["max_total_band_bonus"][var_obj]
        }
    dyn_pen_df = pd.DataFrame(
        [
            {
                "var_obj": var_obj,
                "signal": f"control_action_{var_obj}",
            }
        ]
    )

```

```

        "weight": dyn_pen_cfg["dynamic_penalty_params"][f"control_action"]
        "condition_feature": dyn_pen_cfg["dynamic_penalty_params"][f"condition"]
        "condition_scaled": dyn_pen_cfg["dynamic_penalty_params"][f"condition_scaled"]
        "condition_setpoint": dyn_pen_cfg["dynamic_penalty_params"][f"condition_setpoint"]
    }
    for var_obj in active_var_objs:
        if f"control_action_{var_obj}" in dyn_pen_cfg["dynamic_penalty_params"]:
            # ... (code omitted for brevity) ...
    ]
)
agent_map_df = pd.DataFrame(
    [
        {"agent_name": agent_name, "var_obj": var_obj}
        for agent_name, var_obj in reward_calculator.agent_to_var_obj_map.items()
    ]
)
return principal_df, extra_status_df, band_df, dyn_pen_df, agent_map_df

def build_synthetic_interval(n_steps=decision_steps):
    records = []

    theta_error = np.linspace(0.18, 0.04, n_steps)
    theta_edot = np.linspace(-0.15, -0.03, n_steps)
    theta_I = np.linspace(0.010, 0.040, n_steps)
    theta_u_eff = np.linspace(0.25, 0.08, n_steps)
    theta_u = np.linspace(0.35, 0.10, n_steps)

    x_error = np.linspace(0.03, 0.005, n_steps)
    x_edot = np.linspace(-0.08, -0.01, n_steps)
    x_I = np.linspace(0.003, 0.012, n_steps)
    x_u_eff = np.linspace(0.10, 0.03, n_steps)
    x_u = np.linspace(0.12, 0.04, n_steps)

    theta_delta_u_eff = np.diff(np.concatenate([[theta_u_eff[0]], theta_u_eff]))
    theta_delta_u = np.diff(np.concatenate([[theta_u[0]], theta_u]))
    x_delta_u_eff = np.diff(np.concatenate([[x_u_eff[0]], x_u_eff]))
    x_delta_u = np.diff(np.concatenate([[x_u[0]], x_u]))

    u_total = theta_u_eff + x_u_eff
    delta_u_total = np.diff(np.concatenate([[u_total[0]], u_total]))

    for idx in range(n_steps):
        records.append(
            {
                "error_pendulum_angle": float(theta_error[idx]),
                "derivative_error_pendulum_angle": float(theta_edot[idx]),
                "integral_error_pendulum_angle": float(theta_I[idx]),
                "u_eff_pendulum_angle": float(theta_u_eff[idx]),
                "delta_u_eff_pendulum_angle": float(theta_delta_u_eff[idx]),
                "control_action_pendulum_angle": float(theta_u[idx]),
                "delta_control_action_pendulum_angle": float(theta_delta_u[idx]),
                "error_cart_position": float(x_error[idx]),
                "derivative_error_cart_position": float(x_edot[idx]),
                "integral_error_cart_position": float(x_I[idx]),
                "u_eff_cart_position": float(x_u_eff[idx]),
                "delta_u_eff_cart_position": float(x_delta_u_eff[idx]),
                "control_action_cart_position": float(x_u[idx]),
                "delta_control_action_cart_position": float(x_delta_u[idx]),
                "u_total": float(u_total[idx]),
                "delta_u_total": float(delta_u_total[idx]),
            }
        )

```

```

    )
}

return records

```

1. Reward principal por lazo en el espacio de métricas

Aquí miramos la reward principal en su dominio exacto: métricas ya agregadas por intervalo.

Como la agregación actual es `mean_squared`, la reward principal es simétrica respecto al signo. Por eso los mapas se muestran en términos de magnitud: `|error|` y `|derivative_error|`.

```

In [3]: display(Markdown("## 1A. Mapas de calor de la reward principal"))

fig, axes = plt.subplots(1, len(active_var_objs), figsize=(6 * len(active_var_ob
if len(active_var_objs) == 1:
    axes = [axes]

for ax, var_obj in zip(axes, active_var_objs):
    error_axis = np.linspace(0.0, get_range_abs(var_obj, "e"), 260)
    edot_axis = np.linspace(0.0, get_range_abs(var_obj, "edot"), 260)
    ERROR, EDOT = np.meshgrid(error_axis, edot_axis)
    Z = principal_loop_reward(var_obj, ERROR, EDOT, I_level=0.0, delta_u_eff_lev
    plot_heatmap(
        ax,
        error_axis,
        edot_axis,
        Z,
        x_label=f"|error_{var_obj}|",
        y_label=f"|derivative_error_{var_obj}|",
        title=f"Principal reward - {var_obj}",
        cmap="viridis",
        colorbar_label="principal_reward",
    )

plt.show()

display(Markdown("## 1B. Perfiles de la reward principal"))

fig, axes = plt.subplots(1, len(active_var_objs), figsize=(6 * len(active_var_ob
if len(active_var_objs) == 1:
    axes = [axes]

for ax, var_obj in zip(axes, active_var_objs):
    error_axis = np.linspace(0.0, get_range_abs(var_obj, "e"), 260)
    edot_max = get_range_abs(var_obj, "edot")
    edot_slices = [0.0, 0.25 * edot_max, 0.50 * edot_max, 0.75 * edot_max]

    for edot_level in edot_slices:
        reward_line = principal_loop_reward(var_obj, error_axis, edot_level, I_l
        ax.plot(error_axis, reward_line, label=f"|edot|={edot_level:.3f}")

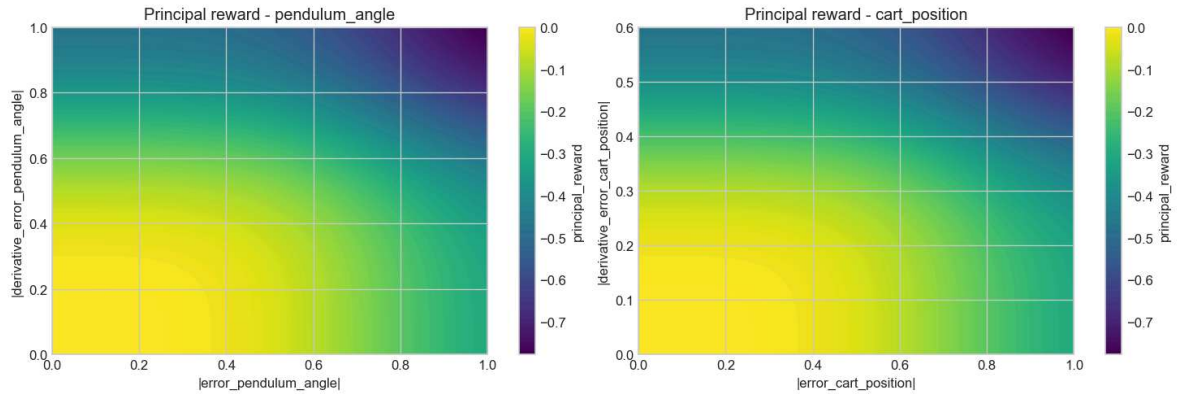
    ax.set_title(f"Perfiles principal reward - {var_obj}")

```

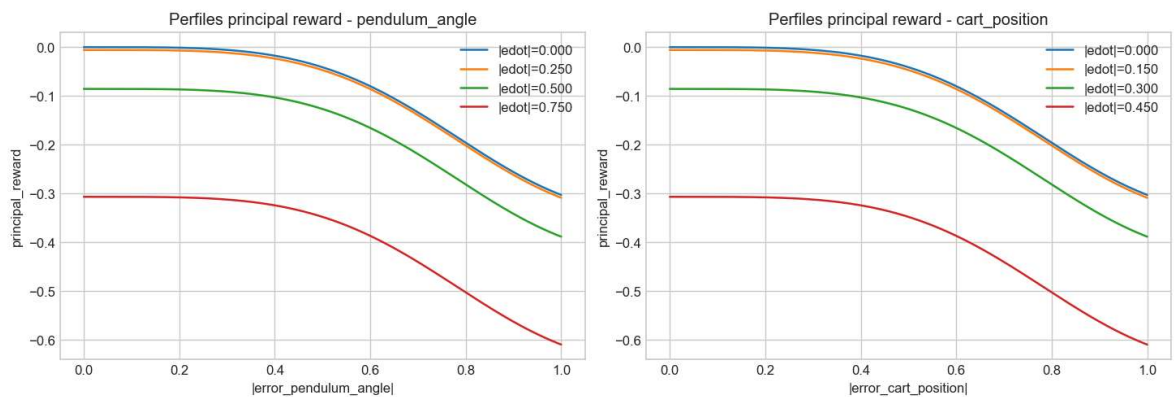
```
ax.set_xlabel(f"|error_{var_obj}|")
ax.set_ylabel("principal_reward")
ax.legend()

plt.show()
```

1A. Mapas de calor de la reward principal



1B. Perfiles de la reward principal



2. Extras activos por lazo

El notebook viejo dedicaba mucha intuición visual a zonas de premio y castigo. Aquí se conserva eso, pero usando las reglas activas reales:

- `bandwidth_bonus` : banda favorable en el error
- `dynamic_penalty` : penalización de la acción de control modulada por el error

```
In [4]: display(Markdown("## 2A. Heatmaps del dynamic penalty + perfil del bandwidth bon

fig, axes = plt.subplots(2, len(active_var_objs), figsize=(6 * len(active_var_ob
if len(active_var_objs) == 1:
    axes = np.array([[axes[0]], [axes[1]]])

for col, var_obj in enumerate(active_var_objs):
    error_axis = np.linspace(0.0, get_range_abs(var_obj, "e"), 260)
    action_axis = np.linspace(0.0, get_range_abs(var_obj, "u"), 260)
    ERROR, ACTION = np.meshgrid(error_axis, action_axis)

    dyn_pen = dynamic_penalty_interval(var_obj, ACTION, ERROR)
    plot_heatmap(
```



```

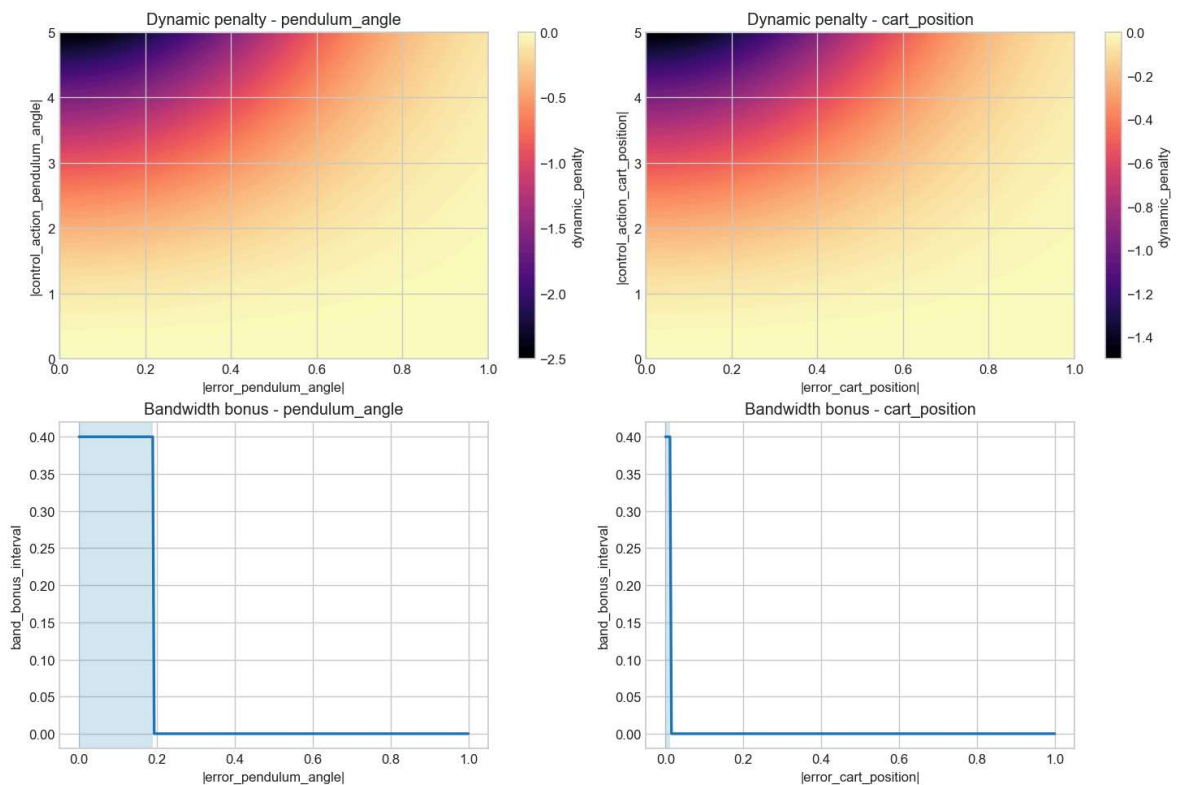
axes[0, col],
error_axis,
action_axis,
dyn_pen,
x_label=f"|error_{var_obj}|",
y_label=f"|control_action_{var_obj}|",
title=f"Dynamic penalty - {var_obj}",
cmap="magma",
colorbar_label="dynamic_penalty",
)

band_bonus = band_bonus_interval(var_obj, error_axis)
band_hi = abs(get_band_range(var_obj)[1])
axes[1, col].plot(error_axis, band_bonus, linewidth=2)
axes[1, col].axvspan(0.0, band_hi, alpha=0.18)
axes[1, col].set_title(f"Bandwidth bonus - {var_obj}")
axes[1, col].set_xlabel(f"|error_{var_obj}|")
axes[1, col].set_ylabel("band_bonus_interval")

plt.show()

```

2A. Heatmaps del dynamic penalty + perfil del bandwidth bonus



3. Reward total proxy en el espacio de comportamiento

Esta es la parte que más se parece al notebook anterior.

Para visualizar cómo se comporta la reward frente a variables "intuitivas", suponemos un intervalo cuasi estacionario: el error medio, la derivada media y la acción media se mantienen aproximadamente constantes dentro de ese intervalo.

Bajo ese supuesto, la reward total por lazo queda:

`principal_loop_reward + bandwidth_bonus + dynamic_penalty`

y puede dibujarse como mapas de calor sobre `|error|` y `|control_action|`.

```
In [5]: display(Markdown("## 3A. Heatmaps de reward total por lazo"))

edot_levels = {}
for var_obj in active_var_objs:
    edot_levels[var_obj] = [0.0, 0.35 * get_range_abs(var_obj, "edot")]

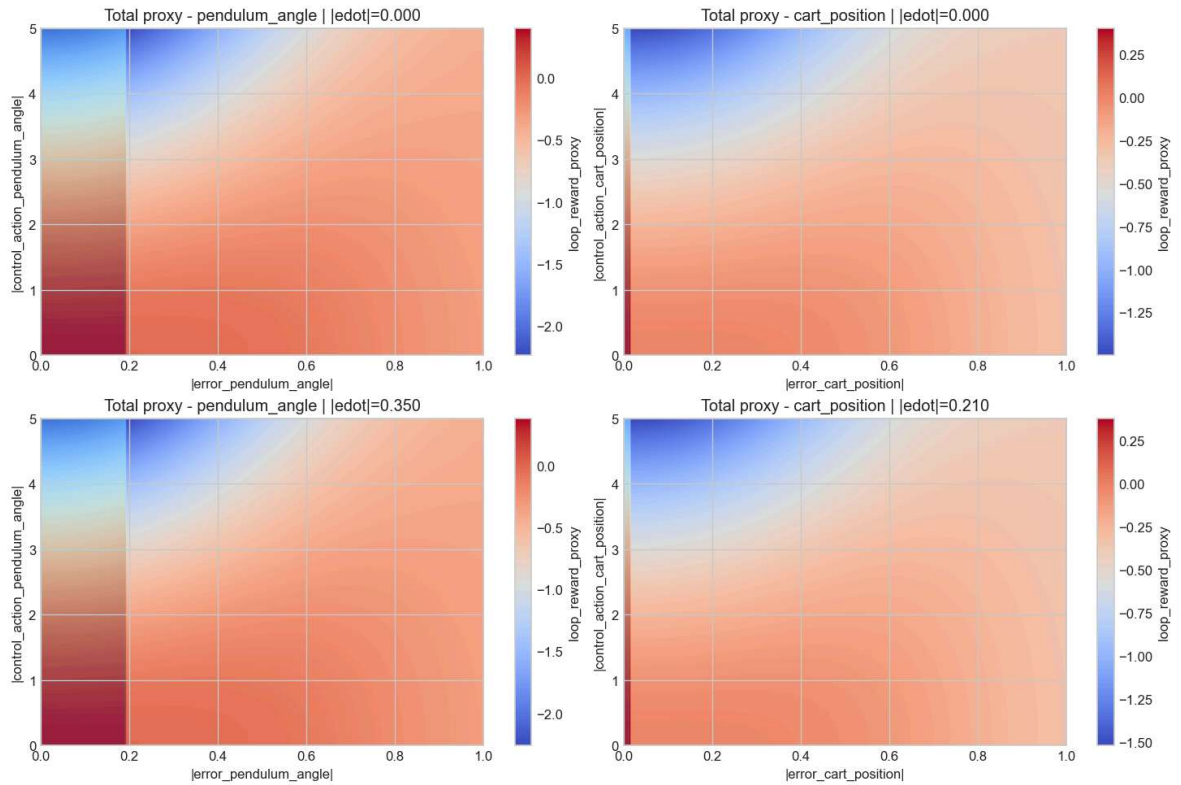
fig, axes = plt.subplots(2, len(active_var_objs), figsize=(6 * len(active_var_objs), 6 * len(active_var_objs)))
if len(active_var_objs) == 1:
    axes = np.array([[axes[0]], [axes[1]]])

for col, var_obj in enumerate(active_var_objs):
    error_axis = np.linspace(0.0, get_range_abs(var_obj, "e"), 260)
    action_axis = np.linspace(0.0, get_range_abs(var_obj, "u"), 260)
    ERROR, ACTION = np.meshgrid(error_axis, action_axis)
    band_hi = abs(get_band_range(var_obj)[1])

    for row, edot_level in enumerate(edot_levels[var_obj]):
        Z = total_loop_reward_proxy(
            var_obj,
            ERROR,
            ACTION,
            edot_level=edot_level,
            I_level=0.0,
            delta_u_eff_level=0.0,
        )
        plot_heatmap(
            axes[row, col],
            error_axis,
            action_axis,
            Z,
            x_label=f"|error_{var_obj}|",
            y_label=f"|control_action_{var_obj}|",
            title=f"Total proxy - {var_obj} | |edot|={edot_level:.3f}",
            cmap="coolwarm",
            colorbar_label="loop_reward_proxy",
        )
        axes[row, col].axvspan(0.0, band_hi, alpha=0.12, color="cyan")

plt.show()
```

3A. Heatmaps de reward total por lazo



```
In [6]: display(Markdown("## 3B. Perfiles de reward total"))

fig, axes = plt.subplots(2, len(active_var_objs), figsize=(6 * len(active_var_ob
if len(active_var_objs) == 1:
    axes = np.array([[axes[0]], [axes[1]]])

for col, var_obj in enumerate(active_var_objs):
    error_axis = np.linspace(0.0, get_range_abs(var_obj, "e"), 260)
    action_axis = np.linspace(0.0, get_range_abs(var_obj, "u"), 260)
    band_hi = abs(get_band_range(var_obj)[1])

    error_slices = [0.0, 0.5 * band_hi, band_hi, 2.0 * band_hi]
    for error_level in error_slices:
        total_line = total_loop_reward_proxy(var_obj, error_level, action_axis,
        axes[0, col].plot(action_axis, total_line, label=f"|error|={error_level}")

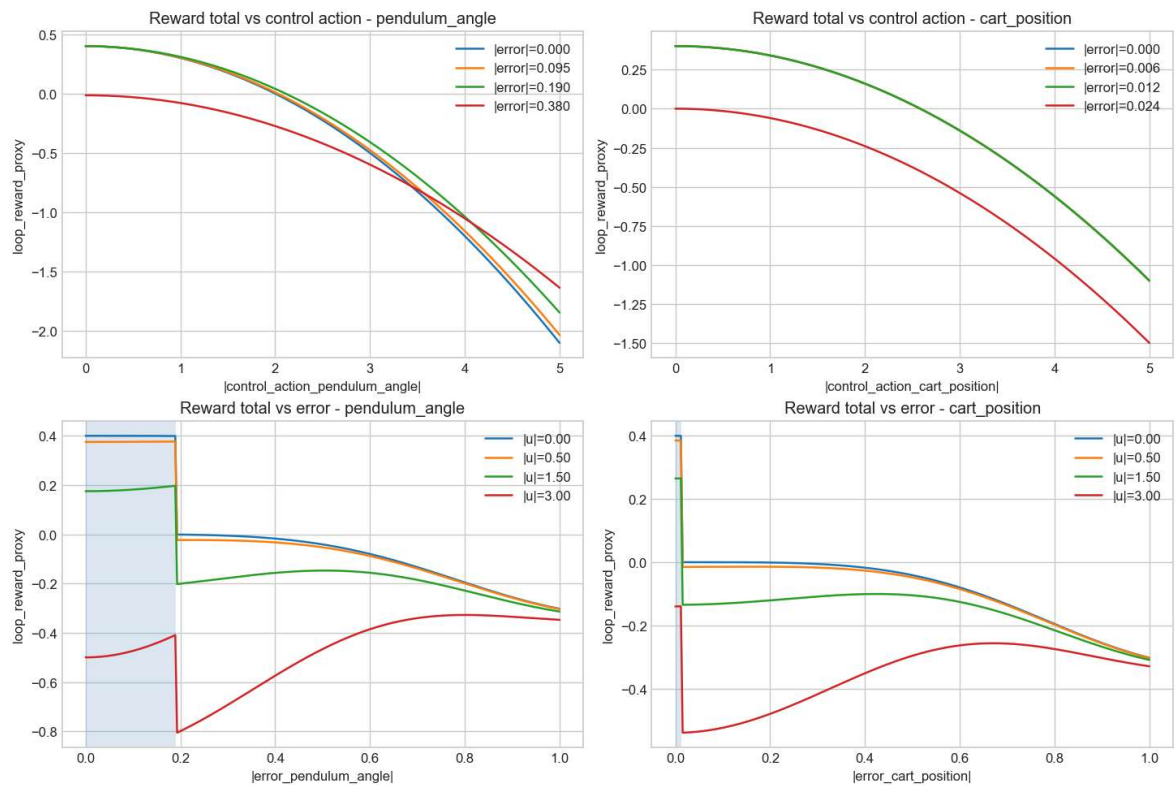
    axes[0, col].set_title(f"Reward total vs control action - {var_obj}")
    axes[0, col].set_xlabel(f"|control_action_{var_obj}|")
    axes[0, col].set_ylabel("loop_reward_proxy")
    axes[0, col].legend()

    action_slices = [0.0, 0.1 * get_range_abs(var_obj, "u"), 0.3 * get_range_abs
    for action_level in action_slices:
        total_line = total_loop_reward_proxy(var_obj, error_axis, action_level,
        axes[1, col].plot(error_axis, total_line, label=f"|u|={action_level:.2f}")

    axes[1, col].axvspan(0.0, band_hi, alpha=0.15)
    axes[1, col].set_title(f"Reward total vs error - {var_obj}")
    axes[1, col].set_xlabel(f"|error_{var_obj}|")
    axes[1, col].set_ylabel("loop_reward_proxy")
    axes[1, col].legend()

plt.show()
```

3B. Perfiles de reward total



4. Resumen declarativo de la reward vigente

Esta tabla deja visibles los pesos, las bandas y los penalties activos que están detrás de los mapas.

```
In [7]: principal_df, extra_status_df, band_df, dyn_pen_df, agent_map_df = build_summary

display(principal_df)
display(extra_status_df)
display(band_df)
display(dyn_pen_df)
display(agent_map_df)
```

	feature	weight	scaled	setpoint
0	L_e	0.35	2.0	0.0
1	L_edot	0.50	3.0	0.0
2	L_l	0.00	1.0	0.0
3	L_delta_u	0.15	0.5	0.0

	block	enabled
0	bandwidth_bonus	True
1	dynamic_penalty	True
2	dynamic_incentive	False
3	goal_bonus_reward	False
4	penalty_delta_var	False
5	penalty_instantaneous_reward	False

	var_obj	band_abs_max	per_step_bonus	interval_bonus_if_full_band	max_total_
0	pendulum_angle	0.190	0.02		0.4
1	cart_position	0.012	0.02		0.4



	var_obj	signal	weight	condition_feature	condition
0	pendulum_angle	control_action_pendulum_angle	0.10	error_pendulum_angle	
1	cart_position	control_action_cart_position	0.06	error_cart_position	



	agent_name	var_obj
0	kp_pendulum_angle	pendulum_angle
1	ki_pendulum_angle	pendulum_angle
2	kd_pendulum_angle	pendulum_angle
3	kp_cart_position	cart_position
4	ki_cart_position	cart_position
5	kd_cart_position	cart_position

5. Check end-to-end con la API actual

Los gráficos anteriores son útiles para diseñar intuición. Aquí cerramos el loop con una pasada real por `MetricProcessing` y `RewardCalculatorBase` para que el notebook siga anclado a la implementación verdadera.

```
In [8]: step_records = build_synthetic_interval()
processed_metrics = metric_processing.process_interval_metrics(step_records)

reward_calculator.reset_episode()
assigned_rewards = reward_calculator.calculate(
    processed_metrics,
    termination_flag="running",
    current_time_sec=simulation_cfg["decision_interval_sec"],
)
```

```
principal_record = reward_calculator._last_principal_record.copy()
extra_record = reward_calculator._last_extra_record.copy()

display(Markdown("### reward_component procesado"))
display(pd.Series(processed_metrics["reward_component"]).sort_index().to_frame("value"))

display(Markdown("### controller rewards"))
display(pd.Series(principal_record["controller_rewards"]).to_frame("value"))

display(Markdown("### extra rewards aplicados"))
display(pd.Series(extra_record).sort_index().to_frame("value"))

display(Markdown("### rewards asignadas a agentes"))
display(pd.Series(assigned_rewards).sort_index().to_frame("value"))
```

reward_component procesado

	value
L_l_cart_position	0.000016
L_l_pendulum_angle	0.000177
L_delta_u_cart_position	0.001289
L_delta_u_pendulum_angle	0.007605
L_delta_u_total	0.060632
L_e_cart_position	0.000364
L_e_pendulum_angle	0.013905
L_edot_cart_position	0.006879
L_edot_pendulum_angle	0.009426
L_u_cart_position	0.000187
L_u_pendulum_angle	0.001195
L_u_total	0.058205
delta_u_eff_cart_position_mean_squared	0.000013
delta_u_eff_pendulum_angle_mean_squared	0.000076
delta_u_total_mean_squared	0.000152
derivative_error_cart_position_mean_squared	0.002476
derivative_error_pendulum_angle_mean_squared	0.009426
error_cart_position_mean_squared	0.000364
error_pendulum_angle_mean_squared	0.013905
integral_error_cart_position_mean_squared	0.000064
integral_error_pendulum_angle_mean_squared	0.000708
u_eff_cart_position_mean_squared	0.004676
u_eff_pendulum_angle_mean_squared	0.029887
u_total_mean_squared	0.058205

controller rewards

	value
pendulum_angle	-0.000273
cart_position	-0.000071

extra rewards aplicados

	value
extra_bonus_band_cart_position	0.120000
extra_bonus_band_pendulum_angle	0.400000
extra_bonus_goal_cart_position	0.000000
extra_bonus_goal_pendulum_angle	0.000000
extra_conditional_dynamic_incentive_cart_position	0.000000
extra_conditional_dynamic_incentive_pendulum_angle	0.000000
extra_conditional_dynamic_penalty_cart_position	-0.000384
extra_conditional_dynamic_penalty_pendulum_angle	-0.004882
extra_penalty_cart_position	0.000000
extra_penalty_instantaneous_cart_position	0.000000
extra_penalty_instantaneous_pendulum_angle	0.000000
extra_penalty_pendulum_angle	0.000000
extra_total_cart_position	0.119616
extra_total_pendulum_angle	0.395118

rewards asignadas a agentes

	value
kd_cart_position	0.119545
kd_pendulum_angle	0.394845
ki_cart_position	0.119545
ki_pendulum_angle	0.394845
kp_cart_position	0.119545
kp_pendulum_angle	0.394845

In []: